CS410 Project

Cindy Nguyen

Prof. Michael Wilson

TA. Amber Shore

CS410 TOP: Data With Python

1/17/2026

Phase 1: Data Acquisition and Cleaning

Project Overview

VocaDB is a fanmade wiki that contains all the information about VOCALOID, primarily songs and setlists for concerts around the world.

VOCALOID and its official concerts, along with special music events are owned and hosted by Crypton Future Media (CFM).

Although CFM has official concerts, there are also many fanmade events as well and other companies that host vocal synth concerts around the world.

VocaDB has an API which is where I'm getting my data from. I want to use this project to predict:

1. What songs will likely show up at the next VOCALOID concert in April (MIKU EXPO 2026)?
2. When will those songs play in the setlist? What is their average position?

Importing VocaDB data from .json files

All of the data pulled from the API is stored in .json format. I created a .py script to pull all of this data from VocaDB, and formatted it a little nicer so it's easier to use/query from.

This is all of the songs from previous concert/event setlists.

In [1]:
```
import pandas as pd
import json
```

```
songs_df = pd.read_json("vocaloid_setlist_songs.json")
songs_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2274 entries, 0 to 2273
Data columns (total 10 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   title              2274 non-null   str
 1   english_title      2274 non-null   str
 2   artist             2274 non-null   str
 3   length_seconds     2274 non-null   int64
 4   setlist_frequency  2274 non-null   int64
 5   avg_setlist_order  2274 non-null   float64
 6   song_type          2274 non-null   str
 7   pv_services        2274 non-null   str
 8   times_favorited    2274 non-null   int64
 9   rating             2274 non-null   int64
dtypes: float64(1), int64(4), str(5)
memory usage: 177.8 KB
```

This looks pretty great! There's no `NaN` values at all, pretty much every field is populated so this cleaning step isn't necessary.

Data Cleansing

Even if there are no `NaN` values, we still have to clean the data. And given that this is numerical data, cleaning it is sort of a different process.

Computing the average setlist order (avg_setlist_order)

One of the data cleansing steps actually has to be done outside of Jupyter notebook since it has to do with how I retrieved my data and computed the average setlist order when creating the dataset itself.

VOCALOID is a big community of fans, full of very talented producers, musicians, and also DJs. This doesn't sound terrible until you realize that these special DJ events can go on for a really long time and mashup tons of songs together.

This is how long the setlist is for the MIKU EXPO Digital Stars 2021 Online event...

No description has been provided for this image

93 songs is a LOT. Fortunately, "MONSTER" was only ever played once at this special event so it's not going to skew the data for other instances of it being played.

However, there were other songs that are much more popular that were in this setlist as well (Melt, Glass Wall, to name a few). These songs would typically show up much earlier in the

setlist across multiple regular concerts, but are played much later at DJ events to build up excitement for the audience.

This is a huge problem, since this will compute the average order to be much higher than it's supposed to be.

It's hard to demonstrate here, but what I'm going to do is exclude any setlists with > 50 songs from my dataset by modifying my Python script.

****I can do this relatively safely for a few reasons****:

1. From my own personal knowledge, most concerts average to around ~30 songs total
   - There are a couple exceptions:
     - Kikuoland (49)
     - Project SEKAI COLORFUL LIVE 4th - Unison (48)
     - VVV MUSIC LIVE concert series (~56)
2. Choosing the number "50" will cover the two main exceptions above, but exclude any super long special events (i.e. DJ sets) which are the MOST likely to skew the value of the average order negatively
3. It's safe to not include any of the VVV MUSIC LIVE concerts (or care at all about VVV MUSIC LIVE)
   - These concerts use voice/character models that would never show up at a MIKU EXPO concert due to licensing reasons UNLESS a major breakthrough happens where a collaboration is viable
     - Nowadays, this is incredibly rare, and in modern times has only happened ONCE with 32ki's song "Mesmerizer" where Kasane Teto made a guest appearance for the JAPAN LIVE TOUR 2025 ~ BLOOMING ~ (and this isn't even a main concert event!)
   - Even if any of the performers had a chance of appearing at MIKU EXPO (e.g. GUMI since she is partially licensed under CFM still), the songs from VVV MUSIC LIVE would never be performed at MIKU EXPO due to licensing reasons

Before/After Results of Removing and Recalculating

Here is the dataset *before* removing all setlists longer than 50 tracks, along with the average positions computed including those long setlists.

In [2]:
```
songs_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2274 entries, 0 to 2273
Data columns (total 10 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   title              2274 non-null   str
 1   english_title      2274 non-null   str
 2   artist             2274 non-null   str
 3   length_seconds     2274 non-null   int64
 4   setlist_frequency  2274 non-null   int64
 5   avg_setlist_order  2274 non-null   float64
 6   song_type          2274 non-null   str
 7   pv_services        2274 non-null   str
 8   times_favorited    2274 non-null   int64
 9   rating             2274 non-null   int64
dtypes: float64(1), int64(4), str(5)
memory usage: 177.8 KB
```

In [3]:

```
# Setting this to 30 so we can actually see the avg_setlist_order be > 1

mask = songs_df["setlist_frequency"] > 30

frequent_songs = songs_df[mask]

frequent_songs
```

Out[3]:

	title	english_title	artist	length_seconds	setlist_frequency	av
64	千本桜	A Thousand Cherry Blossoms	黒うさP feat. 初音ミク	246	37	
73	ルカルカ★ナイトフィーバー	Luka Luka★Night Fever	samfree feat. 巡音ルカ	239	39	
83	メルト	Melt	ryo feat. 初音ミク	256	41	
149	リモコン	Remote Control	じーざすP, WONDERFUL★OPPORTUNITY! feat. 鏡音リン, 鏡音レン	312	31	
152	ワールドイズマイン	World is Mine	ryo, supercell feat. 初音ミク	255	62	
188	???	None		0	75	
268	Tell Your World	None	kz feat. 初音ミク	257	66	

And here is the new dataset *after* removing setlists with more than 50 songs and recalculating the average positions.

In [4]:

```
clean_songs_df = pd.read_json("CLEAN_vocaloid_setlist_songs.json")

clean_songs_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2227 entries, 0 to 2226
Data columns (total 10 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   title              2227 non-null   str
 1   english_title      2227 non-null   str
 2   artist             2227 non-null   str
 3   length_seconds     2227 non-null   int64
 4   setlist_frequency  2227 non-null   int64
 5   avg_setlist_order  2227 non-null   float64
 6   song_type          2227 non-null   str
 7   pv_services        2227 non-null   str
 8   times_favorited    2227 non-null   int64
 9   rating             2227 non-null   int64
dtypes: float64(1), int64(4), str(5)
memory usage: 174.1 KB
```

In [5]:

```
# Setting this to 30 so we can actually see the avg_setlist_order be > 1

mask = clean_songs_df["setlist_frequency"] > 30

clean_frequent_songs = clean_songs_df[mask]

clean_frequent_songs
```

Out[5]:

	title	english_title	artist	length_seconds	setlist_frequency	av
64	千本桜	A Thousand Cherry Blossoms	黒うさP feat. 初音ミク	246	36	
73	ルカルカ★ナイトフィーバー	Luka Luka★Night Fever	samfree feat. 巡音ルカ	239	39	
83	メルト	Melt	ryo feat. 初音ミク	256	41	
149	リモコン	Remote Control	じーざすP, WONDERFUL★OPPORTUNITY! feat. 鏡音リン, 鏡音レン	312	31	
152	ワールドイズマイン	World is Mine	ryo, supercell feat. 初音ミク	255	62	
188	???	None		0	65	
268	Tell Your World	None	kz feat. 初音ミク	257	65	

Results

If you're like me and couldn't tell the difference at first, here's a specific example using "Senbonzakura":

In [6]:
```
frequent_songs.head(1)
```

Out[6]:

	title	english_title	artist	length_seconds	setlist_frequency	avg_setlist_order	song_typ
64	千本桜	A Thousand Cherry Blossoms	黒うさP feat. 初音ミク	246	37	14.27027	Origina

In [7]:
```
clean_frequent_songs.head(1)
```

Out[7]:

	title	english_title	artist	length_seconds	setlist_frequency	avg_setlist_order	song_typ
64	千本桜	A Thousand Cherry Blossoms	黒うさP feat. 初音ミク	246	36	12.361111	Origina

The average setlist position changed! While that's only 2 spots earlier in the setlist for this particular song, that's still easily about 10 minutes of your life waiting for a fan favorite title to play.

2 spots isn't that big of a deal, but for other songs that are even more popular and were included in larger DJ setlists, the changes in the average setlist position could've been much more dramatic.

Songs with 0 seconds in length

So you might've noticed something funky from the output earlier, and it's that there are songs that are 0 seconds in duration.

In [8]:
```
zero_length_songs = clean_songs_df["length_seconds"] == 0
clean_songs_df[zero_length_songs]
```

Out[8]:

	title	english_title	artist	length_seconds	setlist_frequency	avg_setlist_ord
0	STARLIGHT	None	失いP feat. 花隈千冬	0	1	1
5	物語の始まり	Monogatari no Hajimari	失いP feat. 花隈千冬	0	1	6
9	ビーチストーリー	Beach Story	失いP feat. 京町セイカ AI	0	1	10
10	黒いメガネ	Kuroi Megane	失いP feat. 花隈千冬	0	1	11
11	僕の命	Boku no Inochi	失いP feat. 桜乃そら SV	0	1	12
...	...	...	...	...	...	
2147	地球毁灭的那一刻	When the doomsday comes	KIDE, 动点P feat. 星尘	0	1	5
2182	All About That Bass	None	Dharma feat. 京町セイカ (Unknown)	0	1	4
2183	Feel Special	None	ぎゅや feat. 京町セイカ (Unknown)	0	1	5
2187	天城越え	Walk Over Amagi Pass	ぎゅや feat. 京町セイカ (Unknown)	0	1	11
2188	いちご	Ichigo	とりごや feat. 京町セイカ (Unknown)	0	1	12

128 rows × 10 columns

This information is hidden and won't show up on the website, but according to VocaDB, one of these songs is *supposed* to be "FE!N" by Travis Scott.

No description has been provided for this image

As funny as this is, no, CFM has never collaborated with Travis Scott and FE!N is definitely not a Japanese song.

I mentioned it earlier, but VocaDB is a fanmade wiki. And the "fanmade" parts of it really shine through in moments like this.

All humans make mistakes. Some of the mistakes can be incredibly funny (and intentional). Maybe someone's engaging in internet trolling to make my life more inconvenient. Though sometimes, human error and random things *just* happen. The other song titles seem very legitimate - and they are.

****Theories as to why `0` second data happens****:

- A fan is entering in bad data because they think it's funny (it kind of is (it's really inconvenient though))
- Someone entered the data incorrectly but forgot to remove the information from the database itself
- Some of the songs had actual durations but disappeared over time
  - "Monogatari no Hajimari" is supposed to be 4:27 (`267` seconds), but shows up as `0` because the original link to the song doesn't exist anymore (the artist deleted the song or the link to that song vanished for whatever reason)
- Artists delete their old works all the time, and it becomes lost media very quickly in the VOCALOID community to the point that there's a subcommunity that's dedicated to finding the origin of lost songs. If no one archives it, there's no trace of it existing other than basic information that might've been input into the database before it was deleted

Whatever the case may be, we don't really care. I care because I'm a fanatic, but for data processing, cleaning, and the sake of this project, we don't want this data! We're going to get rid of it!

In [9]:
```
# Drop all songs where their length in seconds is 0
# Only keep songs where their length is greater than 0
clean_songs_df = clean_songs_df.loc[clean_songs_df['length_seconds'] > 0]

clean_songs_df
```

Out[9]:

	title	english_title	artist	length_seconds	setlist_frequency	avg_set
1	Love is in Trouble -恋愛主義バージョン-	None	失いP feat. 弦巻マキ AI (Japanese)	236	1	
2	HEADSTRONG -恋愛主義バージョン-	HEADSTRONG -Renai Shugi Version--	失いP feat. 京町セイカ AI	241	1	
3	精神疾患 -恋愛主義バージョン-	Mental Illness -Love Doctrine Version-	失いP feat. 夏色花梨	248	1	
4	愛をつかむ -恋愛主義バージョン-	Ai wo Tsukamu -Renai Shugi Version-	失いP feat. 小春六花 AI	311	1	
6	CAFETERIA -恋愛主義バージョン-	None	失いP feat. ついなちゃん AI	261	1	
...	...	...	...	...	...	
2222	31 secrets	None	毎夜P feat. 重音テト	245	1	
2223	ウルトラトレーラー	Ultra Trailer	マサラダ feat. 重音テトSV	272	1	
2224	吉原ラメント-再来盤-	Yoshiwara Lament -Re-recording-	亜沙 feat. 重音テトSV	224	1	
2225	優しくなれたら	Yasashikunaretara	亜沙 feat. 重音テトSV	202	1	
2226	ODDS & ENDS（テトライブ2025-実演盤-）	ODDS & ENDS (Teto Live 2025 -Live Arrange-)	亜沙, 村田祐一, ASIAN WAVE feat. 重音テトSV	347	1	

2099 rows × 10 columns

In [10]:
```
clean_songs_df.info()
```

```
<class 'pandas.DataFrame'>
Index: 2099 entries, 1 to 2226
Data columns (total 10 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   title              2099 non-null   str
 1   english_title      2099 non-null   str
 2   artist             2099 non-null   str
 3   length_seconds     2099 non-null   int64
 4   setlist_frequency  2099 non-null   int64
 5   avg_setlist_order  2099 non-null   float64
 6   song_type          2099 non-null   str
 7   pv_services        2099 non-null   str
 8   times_favorited    2099 non-null   int64
 9   rating             2099 non-null   int64
dtypes: float64(1), int64(4), str(5)
memory usage: 180.4 KB
```

In [11]:
```
clean_songs_df.describe()
```

Out[11]:

	length_seconds	setlist_frequency	avg_setlist_order	times_favorited	rating
count	2099.000000	2099.000000	2099.000000	2099.000000	2099.000000
mean	234.677465	2.441639	13.449130	29.928537	163.821820
std	79.100950	3.966350	9.764709	55.674499	280.739203
min	41.000000	1.000000	1.000000	0.000000	0.000000
25%	202.000000	1.000000	6.000000	2.000000	12.000000
50%	231.000000	1.000000	11.227273	8.000000	46.000000
75%	263.000000	2.000000	18.000000	30.000000	188.500000
max	2304.000000	65.000000	50.000000	648.000000	2985.000000

In [12]:
```
clean_songs_df.query('length_seconds == 41')
```

Out[12]:

	title	english_title	artist	length_seconds	setlist_frequency	avg_setlist_order	son
2048	またね	Till We Meet Again	ryo, supercell feat. 初音ミク	41	1	41.0	Re

We went from 2227 songs to 2099 songs, removing all 128 songs that were 0 seconds. Now we can see the shortest song is 41 seconds, which makes a lot more sense than a song that practically doesn't exist.

"またね" by ryo is meant to be an outro song and was played at the end of a concert. Concerts used to have outro songs like this one, but it's more likely nowadays that they play

the instrumental of the concert theme to close out for the evening which isn't as easy to register into the database without causing duplicate data.

Songs with no PV services -- Justifications

Some songs have been played at concerts but don't have any PV services. This means that it doesn't have a link to an original source (YouTube, SoundCloud, Spotify, Piapro, NicoNicoDouga, Bilibili, etc.)

There are quite a few of these actually.

In [13]:
```
no_pvs_list = clean_songs_df.query('pv_services == "Nothing"')
no_pvs_list.info()
```

```
<class 'pandas.DataFrame'>
Index: 49 entries, 205 to 2083
Data columns (total 10 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   title              49 non-null     str
 1   english_title      49 non-null     str
 2   artist             49 non-null     str
 3   length_seconds     49 non-null     int64
 4   setlist_frequency  49 non-null     int64
 5   avg_setlist_order  49 non-null     float64
 6   song_type          49 non-null     str
 7   pv_services        49 non-null     str
 8   times_favorited    49 non-null     int64
 9   rating             49 non-null     int64
dtypes: float64(1), int64(4), str(5)
memory usage: 4.2 KB
```

In [14]:
```
no_pvs_list.head(10)
```

Out[14]:

	title	english_title	artist	length_seconds	setlist_frequency	avg_setlist_order	son
205	风兮风兮	None	ilem feat. 洛天依 V4 (萌)	216	1	8.0	(
250	Berlin Calling	None	EHAMIC feat. 結月ゆかり, 石黒千尋	176	1	9.0	
251	月旅	Tsuki Tabi	GYARI feat. 結月ゆかり	282	1	14.0	(
294	香草茶与黑咖啡	Vanilla Tea and Black Coffee	苏逸_Suyi, 动点P feat. 洛天依, 洛天依 V4 日本語 (Sweet)	202	1	2.0	(
382	セカイ	WORLD	Jordan Forester feat. 蒼姫ラピス	258	1	1.0	
383	ビーバー	None	Jordan Forester feat. 蒼姫ラピス	234	1	2.0	
384	ツギハギスタッカート	Patchwork Staccato	Jordan Forester feat. 蒼姫ラピス	251	1	3.0	
385	ジッタードール	Jitter Doll	Jordan Forester feat. 蒼姫ラピス	240	1	4.0	
386	p.h.	None	Jordan Forester feat. 蒼姫ラピス	156	1	5.0	

	title	english_title	artist	length_seconds	setlist_frequency	avg_setlist_order	son
387	シャルル	Charles	Jordan Forester feat. 蒼姫ラピス	228	1	6.0	

A lot of these songs are covers that we can remove safely. However, there are a few originals, remixes, or remasters in there which we definitely want to keep.

****We want to remove these covers that have no PV, but not *ALL* covers from the database for a few reasons****:

1. It's possible that a popular VOCALOID song is *actually* a cover of an originally human song, but fans are simply unaware of it
   - If there is a song registered like this in the database, it would not be a good idea to remove a popular song simply because it's a cover. We want to see all possible songs that will show up, regardless of whether or not it's an original
2. Inspecting closely at the song titles, a vast majority of these songs without any PV are covers that could not be played at an official event due to licensing reasons
   - The list is relatively large (which is why I only display the first 10 entries), but one of the songs is "IDOL" which is originally produced by YOASOBI for the anime "Oshi no Ko". This would definitely NOT show up at a CFM-licensed concert
   - A vast majority of these song covers come from fan events where copyright/licensing isn't as strict, hence why it shows up here
3. You might notice a lot of the top entries are made by someone named "Jordan Forester" using "Aoki Lapis" as the voicebank. While all of their covers are fully VOCALOID songs that didn't originally have a human singer, fans still attribute a particular VOCALOID to be the "original singer"
   - "Patchwork Staccato" is originally sung by Hatsune Miku and has been performed multiple times across different concerts. However, only Miku herself would ever sing that song at an official event. Aoki Lapis would never show up to perform this track since it's not "her song" and the community does not associate her with it
   - A notable exception to this rule would be the Project Sekai concerts, where human-voiced game characters appear on stage and perform *with* the virtual singer. But these tracks are registered separately as a cover and have been performed very sparsely. These covers also have PV services because they're officially licensed by CFM, so they won't show up here

Other songs with no PVs

There's still the fact that while most of the no-PV songs are covers, that doesn't mean that all of them are.

There are a few other categories that show up:

- Original
- Remaster
- MusicPV
- Remix

In [15]:
```
no_pvs_list.query('song_type != "Cover"')
```

Out[15]:

	title	english_title	artist	length_seconds	setlist_frequency	avg_setlist_orde
205	风兮风兮	None	ilem feat. 洛天依 V4 (萌)	216	1	8.00000
251	月旅	Tsuki Tabi	GYARI feat. 結月ゆかり	282	1	14.00000
294	香草茶与黑咖啡	Vanilla Tea and Black Coffee	苏逸_Suyi, 动点P feat. 洛天依, 洛天依 V4 日本語 (Sweet)	202	1	2.00000
469	シリョクケンサ	Eye Examination	40mP feat. GUMI	267	1	17.00000
477	Cross Road	None	19's Sound Factory feat. 初音ミク	238	1	6.00000
627	Fire◎Flower 2011	None	halyosy feat. 鏡音レン	262	1	21.00000
875	8102	None	COP feat. 洛天依 V4 (凝)	276	1	9.00000
895	Nostalogic ["M-E-I-K-O" mix]	None	yuukiss feat. MEIKO	271	3	12.66666
1357	Tornado	None	Various artists	173	2	7.00000
1654	Witness (Vocamerica Mix)	Witness (Vocamerica Mix)	EmpathP feat. DAINA	208	1	11.00000
1713	酒心吻	Liqueur Kiss	LS, Creuzer feat. 乐正绫	288	1	23.00000
1738	ずんだホライずん主題	None	多田彰文 feat.	125	1	37.00000

	title	english_title	artist	length_seconds	setlist_frequency	avg_setlist_orde
	歌(ロングバージョン)		東北ずん子			
1783	碧の輝石	None	中嶋師音 feat. フリモメン	281	1	49.00000
1876	Blessing	None	Colorful Palette feat. various	277	1	10.00000
1882	ヒバナ - Reloaded-	HIBANA - Reloaded-	Colorful Palette feat. various	204	1	9.00000
1883	DAYBREAK FRONTLINE	None	Colorful Palette feat. various	205	1	10.00000
1884	NEO	None	Colorful Palette feat. various	185	1	11.00000
2083	울지 몰라	I Might Cry	고타 feat. SeeU (Korean)	238	1	9.00000

Rather than delete them anyway, my justification to remove or keep some of these tracks are a little bit different than the covers that have no PV services, and some of these cases are actually pretty odd that it's worth explaining what it is.

****Extra notes before removal****:

The lack of PV services implies that these songs are unreachable, undistributed anywhere else officially, and potentially unusable in future events. This may sound like a case of "remove everything", but I'll point out specific entries that I'm going to keep/remove.

1. Originals
   - **Original tracks I'm going to leave alone** on the basis that it may appear in a concert again in the future even if the likelihood is low
     - It's more likely that these tracks will show up at a fan concert, and while I'm not necessarily interested in fan setlists, I am interested in predicting what may show up at the official MIKU EXPO. Fan concerts undeniably demonstrate what fans are interested in, and also show what trends and songs are popular at the

time. CFM definitely accounts for fan behavior/fan favorites when creating events, so it's important to include

2. Remasters
   - **I'll be removing all remasters** from this list **EXCEPT** for "Nostalogic"
     - "Fire◎Flower" is a popular song. However, "Fire◎Flower 2011" has only been played once. This is likely a remastered version produced privately just for that particular concert, that the chance of this audio track ever being reused is low
     - The same also applies to "Eye Examination" - a remastered version was made but never released officially
     - "Nostalogic ["M-E-I-K-O" mix]" I will keep for a couple of reasons:
       A. It has been played more than once
       B. There are multiple different versions of "Nostalogic" - various covers, remasters, remixes, and remasters of those remixes. It's an old and very iconic song for MEIKO, so there are many versions of it that have been redone over the years and the old versions of the song are still played at events. It's best not to remove it
3. Remixes:
   - **I'll be removing all remixes**
   - "Witness (Vocamerica Mix)" is concert-specific to the VOCAMERICA fan concert series. While the original song is available and has PV services, this remix was only for a particular event in the series that it was never officially released
   - All of the remixes by "Colorful Palette feat. various" were for one of the Project Sekai live virtual concerts - in this particular case, they were for the "RESONANCE BEATS" event
     - These tracks are pretty interesting - there's not much information out there about it, but based on the extra notes for "DAYBREAK FRONTLINE" on the wiki, it seems like these remixes were just for this collaboration event between Leo/need and Vivid BAD SQUAD (two in-game music groups) and the remixes were never meant to be released officially anywhere after the event was over
4. MusicPVs
   - **I will remove "Tornado" from the list**
     - Trying to find what this is was kind of complicated. Essentially, it doesn't exist anymore so it should be removed
       - Full explanation: The PV doesn't exist on YouTube, but the song does (not on YouTube - it exists on NicoNicoDouga), but the song used in the PV was just a cover using Megurine Luka. The original song is better known as "Tatsumaki", and the original uses Hatsune Miku as the vocalist. However, the original Miku version was deleted, and the only one that exists now is the version that uses SF-A2 Miki

****Extra information about China-specific tracks (and why I'm not removing them)****:

- "风兮风兮" ["The Wind The Wind"] is performed by Luo Tianyi, who is a Chinese virtual singer licensed by CFM. While this song may be popular in China and might play at a lot of her China-specific concerts, we have no easy way of finding this information and confirming
- Due to China's strict internet rules and the fact that Bilibili is not easily accessible outside of China, this information often never gets reuploaded or discussed on the western side of the fandom
- While it's unlikely that Luo Tianyi or other Chinese virtual idols would perform in America, because they are officially licensed by CFM who hosts MIKU EXPO, there is technically always a non-zero chance that any of them could appear at an event
  - Hatsune Miku and Luo Tianyi have actually performed together multiple times in the past in China, and given that they both have a great relationship with the Chinese fans, it wouldn't be surprising if Luo Tianyi made a surprise appearance at some point in America

Songs with no PV Services -- ACTUAL REMOVAL STEPS (pt 1.)

I spent an entire section justifying what should and shouldn't be removed. I'm fully aware it's incredibly long, and I'm sorry. Anyways, now I'll actually clean up the data.

To recap an incredibly long section...

****We want to remove****:

- All covers with no PVs
- All remixes with no PVs
- "Fire◎Flower 2011"
- "シリョクケンサ"/"Eye Examination"
- "Tornado" Music PV

****We want to keep****:

- Original tracks
- "Nostalogic ["M-E-I-K-O" mix]"
- Chinese-specific tracks (covered by keeping all originals)

In [16]:
```
# I'm going to create a temporary dataframe that holds all the entries I want to re
to_remove_no_pv_items = no_pvs_list.copy()

### Excluding from the list entries we want to keep ###
# Exclude all originals as we want to keep these
to_remove_no_pv_items = to_remove_no_pv_items.query('song_type != "Original"')

# We want to exclude "Nostalogic ["M-E-I-K-O" mix]"
to_remove_no_pv_items = to_remove_no_pv_items.query('title != \'Nostalogic ["M-E-I-
```

Here is our list of items that we want to remove:

In [17]:
```
# Display the list just to confirm we're not going to remove the wrong information

to_remove_no_pv_items.head(5)
```

Out[17]:

	title	english_title	artist	length_seconds	setlist_frequency	avg_setlist_order	son
250	Berlin Calling	None	EHAMIC feat. 結月ゆかり, 石黒千尋	176	1	9.0	
382	セカイ	WORLD	Jordan Forester feat. 蒼姫ラピス	258	1	1.0	
383	ビーバー	None	Jordan Forester feat. 蒼姫ラピス	234	1	2.0	
384	ツギハギスタッカート	Patchwork Staccato	Jordan Forester feat. 蒼姫ラピス	251	1	3.0	
385	ジッタードール	Jitter Doll	Jordan Forester feat. 蒼姫ラピス	240	1	4.0	

In [18]:
```
# And also show that it's not JUST covers in here - remasters, remixes, and music P
to_remove_no_pv_items.query('song_type != "Cover"')
```

Out[18]:

	title	english_title	artist	length_seconds	setlist_frequency	avg_setlist_orde
469	シリョクケンサ	Eye Examination	40mP feat. GUMI	267	1	17
627	Fire◎Flower 2011	None	halyosy feat. 鏡音レン	262	1	21
1357	Tornado	None	Various artists	173	2	7
1654	Witness (Vocamerica Mix)	Witness (Vocamerica Mix)	EmpathP feat. DAINA	208	1	11
1876	Blessing	None	Colorful Palette feat. various	277	1	10
1882	ヒバナ -Reloaded-	HIBANA -Reloaded-	Colorful Palette feat. various	204	1	9
1883	DAYBREAK FRONTLINE	None	Colorful Palette feat. various	205	1	10
1884	NEO	None	Colorful Palette feat. various	185	1	11

In [19]:

```
# There are 39 songs that we want to remove from our actual dataframe
to_remove_no_pv_items.info()
```

```
<class 'pandas.DataFrame'>
Index: 39 entries, 250 to 1884
Data columns (total 10 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   title              39 non-null     str
 1   english_title      39 non-null     str
 2   artist             39 non-null     str
 3   length_seconds     39 non-null     int64
 4   setlist_frequency  39 non-null     int64
 5   avg_setlist_order  39 non-null     float64
 6   song_type          39 non-null     str
 7   pv_services        39 non-null     str
 8   times_favorited    39 non-null     int64
 9   rating             39 non-null     int64
dtypes: float64(1), int64(4), str(5)
memory usage: 3.4 KB
```

Songs with no PV Services -- ACTUAL REMOVAL STEPS (pt. 2)

Now that we have our list of items to remove, we need to remove it from the actual database entirely.

In [20]:
```
# Check to see what songs are on the "no fly" list in our main dataframe, store it
condition = clean_songs_df.isin(to_remove_no_pv_items)

# Anything with all True values is a perfect match to the entries we want to remove
#condition

# Create a new dataframe - this should hold ALL songs but exclude the 39 items on t
all_clean_songs_df = clean_songs_df.drop(to_remove_no_pv_items[condition].index)

all_clean_songs_df.count()
```

Out[20]:
```
title                2060
english_title        2060
artist               2060
length_seconds       2060
setlist_frequency    2060
avg_setlist_order    2060
song_type            2060
pv_services          2060
times_favorited      2060
rating               2060
dtype: int64
```

In [21]:
```
# We can compare the count to the original dataframe we were working with and confi
clean_songs_df.count()
```

Out[21]:
```
title                2099
english_title        2099
artist               2099
length_seconds       2099
setlist_frequency    2099
avg_setlist_order    2099
song_type            2099
pv_services          2099
times_favorited      2099
rating               2099
dtype: int64
```

In [22]:
```
# Another way to confirm is to check if any of the removal list items are in the ne
# The entire dataframe is False so I'm going to use head() to truncate the output
to_remove_no_pv_items.isin(all_clean_songs_df).head()
```

Out[22]:

	title	english_title	artist	length_seconds	setlist_frequency	avg_setlist_order	song_ty
250	False	False	False	False	False	False	Fa
382	False	False	False	False	False	False	Fa
383	False	False	False	False	False	False	Fa
384	False	False	False	False	False	False	Fa
385	False	False	False	False	False	False	Fa

Final Results

Between all of the data cleansing, justifications, and mini VOCALOID history + culture lessons, here is our final dataset.

****Recap on Data Cleanse****:

- Fixed the average setlist order being calculated with bad data
  - Readjusted my Python script to not include setlists larger than 50 as to not corrupt the dataset
  - Recalculated the average setlist order after excluding these setlists so the average turns out the way we expect it to
- Removed all songs `0` seconds in length
- Removed all covers + remixes with no PV services, while keeping some entries due to circumstance/history

In [23]:
```
# Print the dataset one last time to demonstrate the results

all_clean_songs_df
```

Out[23]:

	title	english_title	artist	length_seconds	setlist_frequency	avg_set
1	Love is in Trouble -恋愛主義バージョン-	None	失いP feat. 弦巻マキ AI (Japanese)	236	1	
2	HEADSTRONG -恋愛主義バージョン-	HEADSTRONG -Renai Shugi Version--	失いP feat. 京町セイカ AI	241	1	
3	精神疾患 -恋愛主義バージョン-	Mental Illness -Love Doctrine Version-	失いP feat. 夏色花梨	248	1	
4	愛をつかむ -恋愛主義バージョン-	Ai wo Tsukamu -Renai Shugi Version-	失いP feat. 小春六花 AI	311	1	
6	CAFETERIA -恋愛主義バージョン-	None	失いP feat. ついなちゃん AI	261	1	
...	...	...	...	...	...	
2222	31 secrets	None	毎夜P feat. 重音テト	245	1	
2223	ウルトラトレーラー	Ultra Trailer	マサラダ feat. 重音テトSV	272	1	
2224	吉原ラメント-再来盤-	Yoshiwara Lament -Re-recording-	亜沙 feat. 重音テトSV	224	1	
2225	優しくなれたら	Yasashikunaretara	亜沙 feat. 重音テトSV	202	1	
2226	ODDS & ENDS（テトライブ2025-実演盤-）	ODDS & ENDS (Teto Live 2025 -Live Arrange-)	亜沙, 村田祐一, ASIAN WAVE feat. 重音テトSV	347	1	

2060 rows × 10 columns

Thank you!

I'm tired... time for a break...

Phase 2: Exploratory Data Analysis (EDA)

Overview

Now that we've handled getting our data and the big primary cleaning steps, let's continue!

We now have our cleaned-up dataframe. I'll display it again below just to confirm it still exists, but now that we have it we want to start visualizing our data.

In [24]:
```
# Display all_clean_songs_df to confirm its existence
all_clean_songs_df
```

Out[24]:

	title	english_title	artist	length_seconds	setlist_frequency	avg_set
1	Love is in Trouble -恋愛主義バージョン-	None	失いP feat. 弦巻マキ AI (Japanese)	236	1	
2	HEADSTRONG -恋愛主義バージョン-	HEADSTRONG -Renai Shugi Version--	失いP feat. 京町セイカ AI	241	1	
3	精神疾患 -恋愛主義バージョン-	Mental Illness -Love Doctrine Version-	失いP feat. 夏色花梨	248	1	
4	愛をつかむ -恋愛主義バージョン-	Ai wo Tsukamu -Renai Shugi Version-	失いP feat. 小春六花 AI	311	1	
6	CAFETERIA -恋愛主義バージョン-	None	失いP feat. ついなちゃん AI	261	1	
...	...	...	...	...	...	
2222	31 secrets	None	毎夜P feat. 重音テト	245	1	
2223	ウルトラトレーラー	Ultra Trailer	マサラダ feat. 重音テトSV	272	1	
2224	吉原ラメント-再来盤-	Yoshiwara Lament -Re-recording-	亜沙 feat. 重音テトSV	224	1	
2225	優しくなれたら	Yasashikunaretara	亜沙 feat. 重音テトSV	202	1	
2226	ODDS & ENDS（テトライブ2025-実演盤-）	ODDS & ENDS (Teto Live 2025 -Live Arrange-)	亜沙, 村田祐一, ASIAN WAVE feat. 重音テトSV	347	1	

2060 rows × 10 columns

Data Visualizations

I'll be using seaborn and matplotlib to create all of my data visuals.

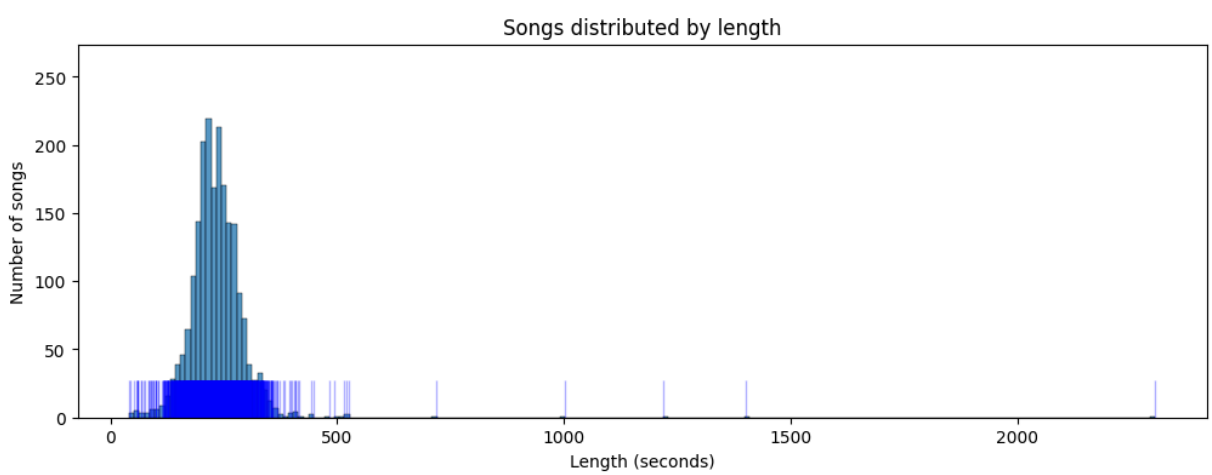
In [25]:
```
import seaborn as sns
import matplotlib.pyplot as plt
```

Graphing Songs Based On Length

Music is music - there are all kinds of different genres, artists, languages, but universally they all share one thing in common: all songs must have some sort of length.

This database has a lot of songs, and we can plot that into a histogram to see what time amounts are the most common. I also use a rug plot to show how clustered/dispersed the songs are in length.

In [26]:
```
# Create figure
plt.figure(figsize=(12,4))

# Create histogram chart
sns.histplot(data=all_clean_songs_df, x="length_seconds", bins=200)

# Create rug plot to show dispersion of data
sns.rugplot(all_clean_songs_df['length_seconds'], height=0.1, color='blue', alpha=0

plt.title("Songs distributed by length")
plt.xlabel("Length (seconds)")
plt.ylabel("Number of songs")

plt.show()
```



In [27]:
```
all_clean_songs_df[all_clean_songs_df["length_seconds"] > 600]
```

Out[27]:

	title	english_title	artist	length_seconds	setlist_frequency	avg_setlist_order
254	ボーカロイドたちがただﾃｯﾃｰﾃﾚｯﾃｰするだけ	Vocaloids just do tettey-terettey only	GYARI feat. various	1219	1	19.0
672	ボーカロイドたちがただ叫ぶだけ	Vocaloids yelling only	GYARI feat. 鏡音リン, 鏡音レン	1401	1	23.0
784	Sight, Noise, Life and the Earth	None	きくお feat. 初音ミクV3 (English)	2304	1	21.0
1587	Alice in Musicland	None	OSTER project feat. various	718	2	14.5
2067	천 년의 시	Poem of a Thousand Years	상록수 feat. SeeU (Korean)	1002	1	20.0

What are those outliers?

Over 2000 minutes is absurd and I'm fully aware of this. The longest song is "Sight, Noise, Life and the Earth" by Kikuo, which at 2304 seconds is ~38.4 minutes.

I was really worried that I didn't clean the data thoroughly enough and only noticed something crazy now, but actually, it's perfectly fine that these exist. They've only been played once or twice at concerts, and it's much more likely that during the concerts themselves only a small fraction of the full songs were played. With the way VocaDB enters information, the database doesn't tell us or give us much warning if this is the case, but it's safe to assume so. I would hope a 30 minute song *wouldn't* be playing during a concert.

These are also real songs (yes, even "Vocaloids yelling only" or the other one by GYARI), and if they were played at a concert at one point that does show some level of interest in those titles.

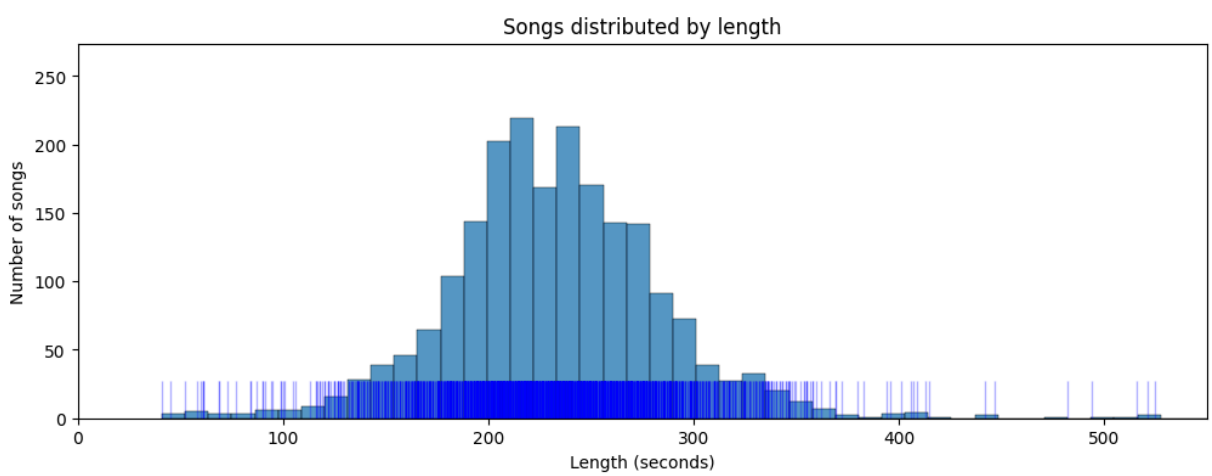
Either way, I won't remove it from the data. What I am going to do is remake the same graph, but limit the range of what we're looking at to temporarily ignore these outliers without deleting them entirely from our dataframe.

****This feels like I'm creating misleading graphics, but I'll try to justify why this is okay****:

- These outliers have no chance of appearing at an official MIKU EXPO due to the nature of the songs being way too long
- The songs in this list likely wouldn't appear at a MIKU EXPO due to the content of the songs
  - Kikuo's music is disturbing and not kid-friendly (CFM often avoids playing songs that are too explicit)
  - GYARI's music is jazzy and has no lyrics so this isn't concert-friendly
  - Any song with SeeU won't play at MIKU EXPO since she's Korean, a retired voicebank, and there was controversy in 2014 regarding her voice provider (Dahee from K-pop girl group "GLAM")

Here's the adjusted graphic below to focus primarily on where most of the data is, ignoring the outliers:

In [28]:
```
# Create the figure
plt.figure(figsize=(12,4))

# Create the histogram
sns.histplot(data=all_clean_songs_df, x="length_seconds", bins=200)

# Create the rug plot to show the concentrations of data
sns.rugplot(all_clean_songs_df['length_seconds'], height=0.1, color='blue', alpha=0

# Limit the diagram up to 550 seconds to include the data we need and exclude the o
plt.xlim(0, 550)

plt.title("Songs distributed by length")
plt.xlabel("Length (seconds)")
plt.ylabel("Number of songs")

plt.show()
```



After condensing the results and focusing only up to 550 seconds, we can see a lot of the songs are somewhere between ~170-300 seconds.

This makes sense - most concert songs would be around this length to encourage fan participation and allow fans to enjoy songs without it feeling too short or too long.

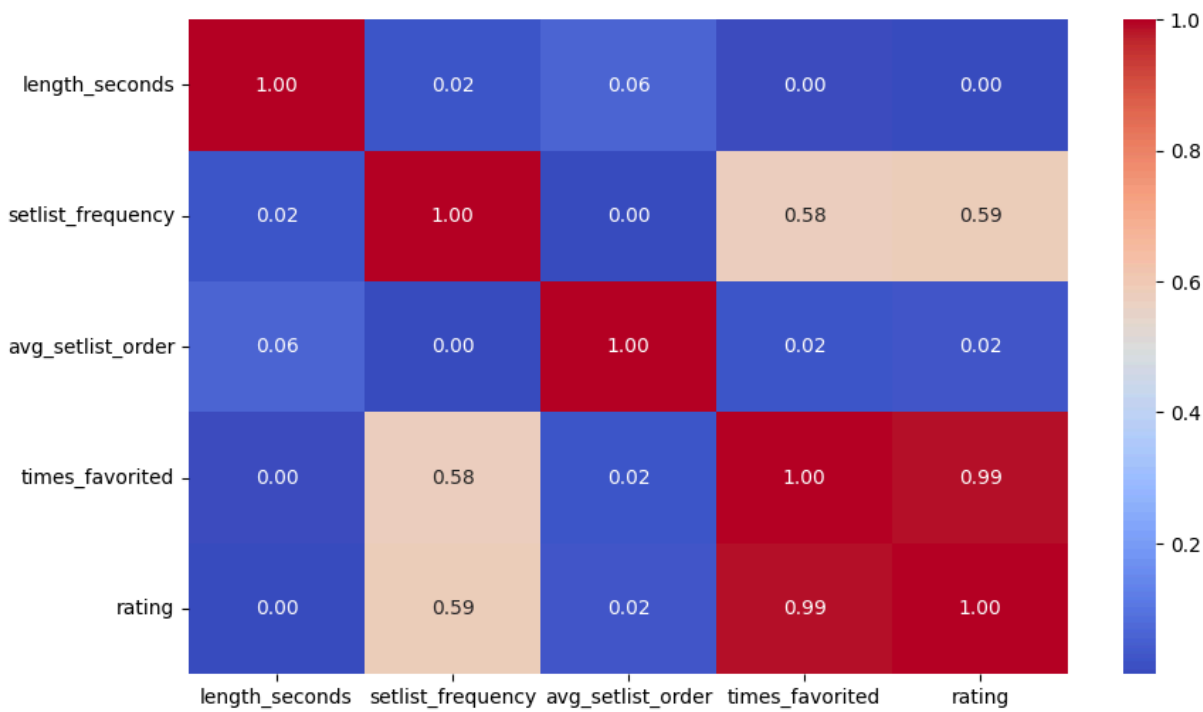
Heat Maps to Demonstrate Correlation

As an older VOCALOID fan, it's really easy for me to say "if a song is more popular, obviously it's more likely to play at a concert!" I mean, it makes sense, right? If a song is more popular, naturally the fans are more likely to want to see it live. This project was maybe the first time I had to really think "wait, is that actually true?" Intuitively and with what I know, it is true, but let's actually prove it so I'm not lying and spreading fake fandom news.

A great way to visualize correlations between numerical data is by using heat maps.

In [29]:
```
# Create the figure
plt.figure(figsize=(10,6))

# Create a correlation matrix for the songs
song_corr_matrix = all_clean_songs_df.corr(numeric_only=True)

# Make the figure a heat map
sns.heatmap(data=song_corr_matrix, annot=True, cmap="coolwarm", fmt=".2f")
```

Out[29]: <Axes: >



This is simply *fascinating*. This is actual proof that there is a correlation between how popular a song is compared to how frequently it's been played at concerts in the past. Initially, I was

expecting it to be not much, but 0.58-0.59 is a relatively undeniable correlation between the two and much higher than I expected!

With this kind of knowledge, it'll be easier to predict what could play at the next MIKU EXPO, but I also want to point out that this map also demonstrates what songs the fandom really values. What songs fans care about, what songs they really like, iconic hits, nostalgic tunes... which songs continue to persist as time passes and as trends change? VOCALOID is a very vast community, and seeing that there still are community favorites despite how large and old the fanbase has become is very touching.

PV Services and Popularity

Some history about PV services and why there are so many

This is going to be a long one. Just know that "PV" stands for "promotional video".

At the start of VOCALOID and it's conception, it was a niche and obscure thing. No one really knew what it was - an anime character from some TV show? An instrument? A robot? How human is it? Is it human at all? Even now, people who are new to the concept of vocal synths ponder the same questions and in some ways, it is still a niche community (at least, in terms of the technology that VOCALOIDs use).

Before the age of generative AI and fake AI voice models, all we had were simple consonant-vowel vocal synths such as these. We had no clue they would take the world by storm and be able to perform live concerts years later. But during its humble beginnings, a small community was surely forming around these virtual idols, and it began (mostly) on NicoNicoDouga.

NicoNicoDouga (NND) is Japan's version of YouTube. YouTube wasn't banned nor was it restricted in Japan, but NND was more commonly used. Its early days are remembered for having a lot of creative content related to fandoms, and the website was where the primary VOCALOID community was during the late 2000's and early 2010's.

As VOCALOID grew, so did its fanbase. Soon, VOCALOID was becoming popular in America that songs were redistributed from NND to YouTube, and YouTube slowly became the main place to view, distribute songs, and interact with the fandom as a whole. Nowadays, VOCALOID is popular across multiple social media platforms and there are many places to listen to the music.

But enough history, let's move on to the data.

Data Explosions and Strange Calculations

As mentioned above, there are a lot of PV services you can use to listen to VOCALOID music nowadays even if historically, it all really began on NND.

I want to know a couple things about the different PV services that the fandom uses:

1. Which PV services have the highest average song rating?
2. Which PV services are used, and which ones have been used the most?

In [30]:
```
all_clean_songs_df.tail(5)
```

Out[30]:

	title	english_title	artist	length_seconds	setlist_frequency	avg_setlist_order
2222	31 secrets	None	毎夜P feat. 重音テト	245	1	15.0
2223	ウルトラトレーラー	Ultra Trailer	マサラダ feat. 重音テト SV	272	1	16.0
2224	吉原ラメント-再来盤-	Yoshiwara Lament -Re-recording-	亜沙 feat. 重音テト SV	224	1	17.0
2225	優しくなれたら	Yasashikunaretara	亜沙 feat. 重音テト SV	202	1	18.0
2226	ODDS & ENDS（テトライブ2025-実演盤-）	ODDS & ENDS (Teto Live 2025 - Live Arrange-)	亜沙, 村田祐一, ASIAN WAVE feat. 重音テト SV	347	1	19.0

You may see something funky here, and it's that some of the values in "pv_services" are a comma-separated list of names. Normally, most people wouldn't think much of it, but when it comes to dealing with data directly it's kind of awful.

It sounds hilarious but we're gonna have to explode (***boom***) our data.

We need to split up these comma-separated lists and have each PV service have its own individual song listing so we can count all the occurrences of each service accurately.

In [31]:
```
# Make a copy of the original dataframe
exploded_songs = all_clean_songs_df.copy()

# exploded_songs is an exact copy of all_clean_songs_df
# We now want to split up those comma-separated lists and split based on the commas
exploded_songs['pv_services'] = all_clean_songs_df['pv_services'].str.split(",")

# We now EXPLODE!!! For each song, there will now be extra entries if there are mul
exploded_songs = exploded_songs.explode('pv_services')

# Because of the way we split the lists earlier, we will have trailing whitespace.
exploded_songs['pv_services'] = exploded_songs['pv_services'].str.strip()

exploded_songs.tail(10)
```

Out[31]:

	title	english_title	artist	length_seconds	setlist_frequency	avg_setlist_order
2221	好きな惣菜発表ドラゴン	Favorite-Daily-Dish-Announcing Dragon	ンバヂ feat. 重音テト	122	1	14.0
2221	好きな惣菜発表ドラゴン	Favorite-Daily-Dish-Announcing Dragon	ンバヂ feat. 重音テト	122	1	14.0
2222	31 secrets	None	毎夜P feat. 重音テト	245	1	15.0
2223	ウルトラトレーラー	Ultra Trailer	マサラダ feat. 重音テト SV	272	1	16.0
2223	ウルトラトレーラー	Ultra Trailer	マサラダ feat. 重音テト SV	272	1	16.0
2223	ウルトラトレーラー	Ultra Trailer	マサラダ feat. 重音テト SV	272	1	16.0
2224	吉原ラメント-再来盤-	Yoshiwara Lament -Re-recording-	亜沙 feat. 重音テト SV	224	1	17.0
2224	吉原ラメント-再来盤-	Yoshiwara Lament -Re-recording-	亜沙 feat. 重音テト SV	224	1	17.0
2225	優しくなれたら	Yasashikunaretara	亜沙 feat. 重音	202	1	18.0

	title	english_title	artist	length_seconds	setlist_frequency	avg_setlist_order
			テトSV			
2226	ODDS & ENDS (テトライブ2025-実演盤-)	ODDS & ENDS (Teto Live 2025 - Live Arrange-)	亜沙, 村田祐一, ASIAN WAVE feat. 重音テトSV	347	1	19.0

As you can see, for any songs that have multiple PV services they are now listed multiple times with the PV services listed individually in each entry.

Creating this `exploded_songs` dataframe was just an intermediate step, because now we're going to start performing aggregate operations to calculate the average, median, and standard deviation for our data

Calculating the Mean/Average

Here, I'll be grouping our `exploded_songs` by `pv_services`, so this way we can obtain the average `rating` per PV service.

In [32]:
```
# Group by pv_services
songs_groupby_service = exploded_songs.groupby('pv_services')

# Calculate the mean for each of these services
songs_service_mean = songs_groupby_service['rating'].mean()

songs_service_mean
```

Out[32]:
```
pv_services
Bandcamp         161.278075
Bilibili         195.862434
NicoNicoDouga    213.653307
Nothing            8.600000
Piapro           267.321429
SoundCloud       292.613260
Vimeo            293.363636
Youtube          174.872699
Name: rating, dtype: float64
```

Because we grouped by `exploded_songs` and calculated the service ratings through that grouping, something odd happened. It has to do with the data explosion that I did earlier:

> We need to split up these comma-separated lists and have each PV service have its own individual song listing so we can count all the occurrences of each service accurately.

> As you can see, for any songs that have multiple PV services they are now listed multiple times with the PV services listed individually in each entry.

This will affect our numbers very heavily. Having multiple entries for the same song/service will bloat our values a LOT. Moving forward, we have to be careful about how we use and interpret these values.

Because we know the data explosion will bloat our values and cause some issues, let's calculate the *actual* average overall rating of all songs. We'll use the original `all_clean_songs_df` to do this.

In [33]:
```
# Calculate average rating of ALL songs across all PV services
# If we used the exploded version (exploded_songs), this would cause multiple entri
songs_rating_mean = all_clean_songs_df['rating'].mean()
print(songs_rating_mean)
```

```
166.87233009708737
```

Below, notice how `songs_rating_mean` (overall) is much lower than `songs_service_mean` (exploded, grouped by PV services).

Based off of this, this is implying that Bandcamp and "Nothing" have song ratings below average, while every other platform such as Vimeo or SoundCloud are nearly double the average.

This won't add up with what we'll learn about those platforms later on in this section.

In [34]:
```
print(f"Exploded, grouped PV services mean:\n{songs_service_mean}\n")
print(f"Song rating average overall:\n{songs_rating_mean}")
```

```
Exploded, grouped PV services mean:
pv_services
Bandcamp         161.278075
Bilibili         195.862434
NicoNicoDouga    213.653307
Nothing            8.600000
Piapro           267.321429
SoundCloud       292.613260
Vimeo            293.363636
Youtube          174.872699
Name: rating, dtype: float64

Song rating average overall:
166.87233009708737
```

Calculating the Median

We're seeing some strange things with the `exploded_songs`. We will also do the same here - we will avoid using the `exploded_songs` dataframe to grab the median value of the song ratings, but we'll still use it to calculate the exploded median to show how exploding the data caused some issues with our numbers.

In [35]:
```
# Calculate the overall median song rating using all_clean_songs_df
songs_rating_median = all_clean_songs_df['rating'].median()
print(songs_rating_median)
```

```
49.0
```

In [36]:
```
# Calculate the median song rating using the exploded_songs that are grouped by pv_
songs_service_median = songs_groupby_service['rating'].median()
songs_service_median
```

Out[36]:
```
pv_services
Bandcamp          62.0
Bilibili          72.0
NicoNicoDouga     80.0
Nothing            7.0
Piapro           130.5
SoundCloud        77.0
Vimeo             77.0
Youtube           54.0
Name: rating, dtype: float64
```

Once again, below, you'll see that the values are very different! This is showing that pretty much every PV service has ratings well above the median (except for "Nothing").

In [37]:
```
print(f"Exploded, grouped PV services median:\n{songs_service_median}\n")
print(f"Song rating median overall:\n{songs_rating_median}")
```

```
Exploded, grouped PV services median:
pv_services
Bandcamp          62.0
Bilibili          72.0
NicoNicoDouga     80.0
Nothing            7.0
Piapro           130.5
SoundCloud        77.0
Vimeo             77.0
Youtube           54.0
Name: rating, dtype: float64

Song rating median overall:
49.0
```

Calculating the Standard Deviation

So what's really going on here?

The data explosion we had to do earlier was necessary, but it does cause a lot of craziness with the values it produces. That's not really the explosion's fault necessarily, that just comes down to how numbers work.

We have to be VERY careful if we do decide to do calculations based on the exploded dataframe, and even though I've spent a lot of this section essentially booing the exploded data for being inaccurate and misleading, it's still very useful and informative about the songs under specific PV services.

There's also one particular thing that's interesting about all this, and it's the standard deviation.

In [38]:
```
# Calculate the standard deviation based on exploded_songs
songs_service_std = songs_groupby_service['rating'].std()
songs_service_std
```

Out[38]:
```
pv_services
Bandcamp         225.503231
Bilibili         290.989524
NicoNicoDouga    313.753148
Nothing            8.030497
Piapro           302.785602
SoundCloud       459.104726
Vimeo            440.528381
Youtube          287.084848
Name: rating, dtype: float64
```

In [39]:
```
# Calculate the overall standard deviation based on all_clean_songs_df
songs_rating_std = all_clean_songs_df['rating'].std()
print(songs_rating_std)
```

```
282.4989192024742
```

For convenience, the different standard deviations are listed below so they're easily comparable.

The standard deviation will vary pretty widely since it's based on ratings, and ratings vary a lot in our data. A song could have very low ratings near 0, but in the same vein another song could have ratings in the hundreds or thousands. There's a lot of variety here!

What's interesting is even in the exploded data, the standard deviation between both dataframes is nearly the exact same for YouTube. It'll be really cool once we get to the next part of this section where we finally start visualizing our data, because we'll get to actually see that YouTube is pretty average in terms of its ratings. It's fascinating watching the numbers prove themselves.

In [40]:
```
print(f"Exploded, grouped PV services standard deviation:\n{songs_service_std}\n")
print(f"Song rating standard deviation overall:\n{songs_rating_std}")
```

```
Exploded, grouped PV services standard deviation:
pv_services
Bandcamp         225.503231
Bilibili         290.989524
NicoNicoDouga    313.753148
Nothing            8.030497
Piapro           302.785602
SoundCloud       459.104726
Vimeo            440.528381
Youtube          287.084848
Name: rating, dtype: float64

Song rating standard deviation overall:
282.4989192024742
```

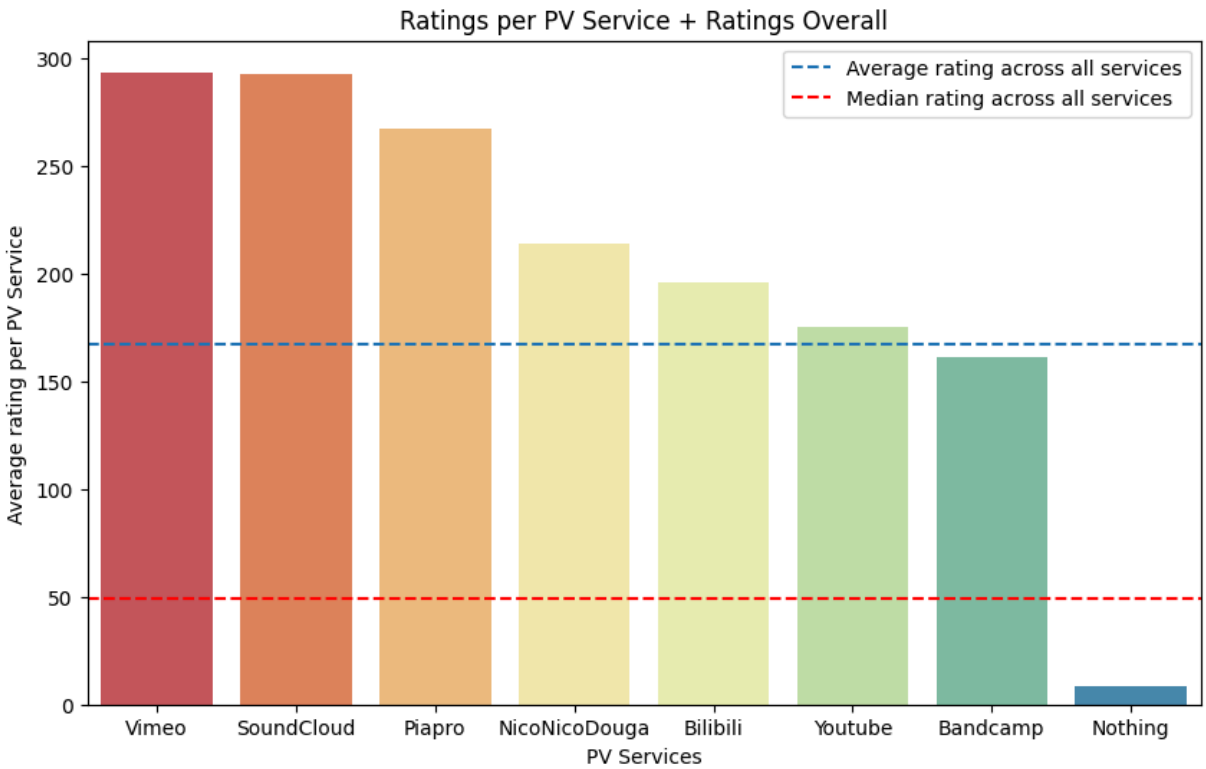
Visualizing Ratings Against PV Services

Now that I've talked about the math part for long enough, it's time to plot all of our data so we can see the results.

What will happen:

- I'll be plotting the average rating based on each PV service
- I'll also place dotted lines to show the actual average/median rating across all songs so we can see how the true average/median compares to each PV service
- Comments in the code will be included to explain everything, and extra observations will be made below the graph itself

In [41]:
```
# Reset the index - this is so we can actually plot the data
songs_service_mean = songs_service_mean.reset_index("pv_services")

# Sort the values by rating so the data looks more cohesive
songs_service_mean = songs_service_mean.sort_values(by="rating", ascending=False)

# Create the figure
plt.figure(figsize=(10,6))

plot = sns.barplot(data=songs_service_mean, x="pv_services", y="rating", hue="pv_se
plot.axhline(songs_rating_mean, linestyle="--", label="Average rating across all se
plot.axhline(songs_rating_median, linestyle="--", label="Median rating across all s
#plot.axhline(songs_rating_std, linestyle="--", label="Standard deviation of rating
plot.legend() # Create legend to make dotted lines clear

plt.title("Ratings per PV Service + Ratings Overall")
plt.xlabel("PV Services")
plt.ylabel("Average rating per PV Service")

plt.show()
```



I mentioned it earlier but seeing YouTube almost have the average overall rating is pretty crazy! It is barely over the average rating across all services, even after exploding the data which I would've expected to cause the data to be much more skewed towards the upper bound.

I talk about it during Phase 1, but Bilibili is a Chinese social media app and there are Chinese producers who make VOCALOID songs. However, I never really mentioned Piapro all that much. Piapro is CFM's official website that allows users to post their original art or music of VOCALOID characters. It's more commonly used when producers want to release their instrumental/track in lossless audio quality, or when an artist who made visuals for a music video wants to repost the visuals in full quality for fans to enjoy.

"Nothing" is expected - there are very few songs that don't have PV services with high ratings. It would be strange to have a popular song not be accessible at all. I'm sure it's happened before but it's a rare case and it makes sense the average looks the way it does for that column.

What *isn't* expected is Vimeo and SoundCloud having such a high average rating. I don't want to be mean, but who uses Vimeo as their main video-watching service? Same with SoundCloud. SoundCloud is popular with independent producers but it's not as common as it used to be and it is also not commonly used among VOCALOID fans or producers.

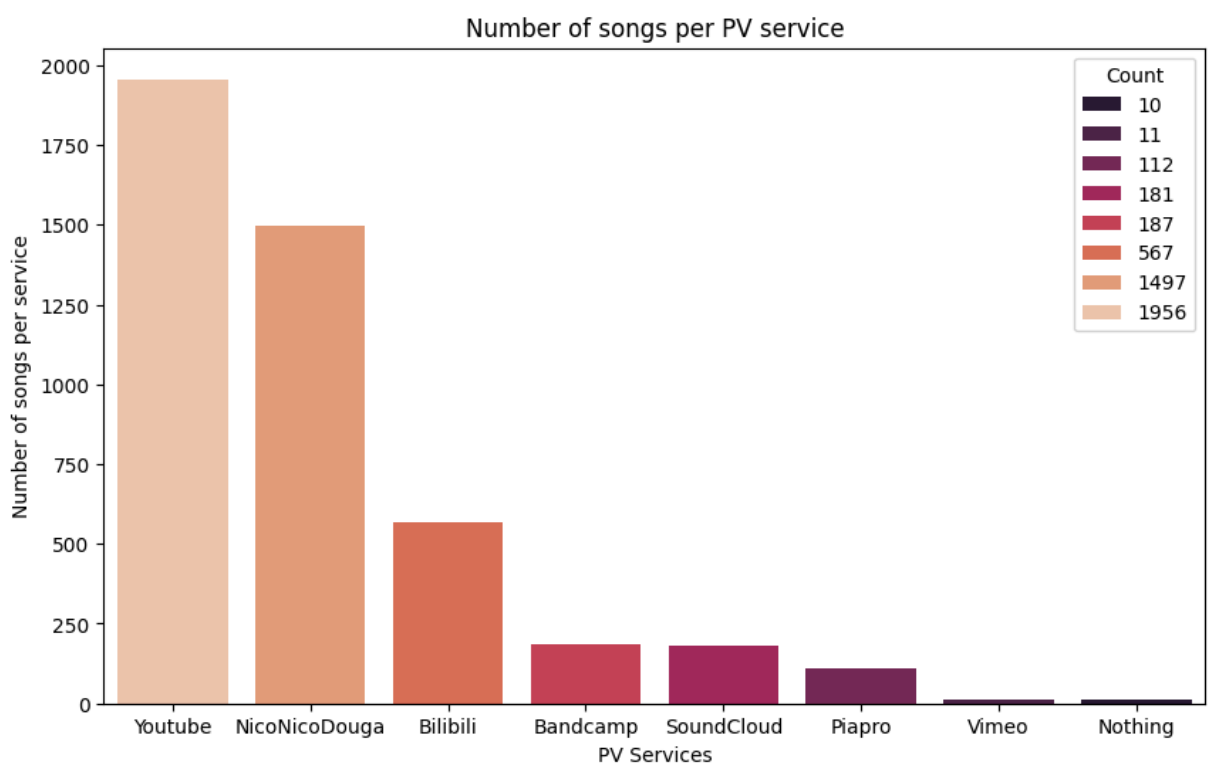
Distribution of Songs per PV Service

The averages for Vimeo and SoundCloud don't really add up... This visualization was probably (most likely) painfully obvious to do at the start. But I wanted to highlight why the

averages turned out the way they did before I spoiled the reasoning why from the very beginning

Here's another chart (that I probably should've done since the start)! Here I'm comparing each PV service with how many songs are hosted on each service. The results are pretty wild.

In [42]:
```
# Retrieve the count for every PV service
songs_service_count = songs_groupby_service['pv_services'].count()

# Sort the number of songs for each service
songs_service_count = songs_service_count.sort_values(ascending=False)

# Reset the index for the count and give it the name "Count" so we can refer to the
songs_service_count = songs_service_count.reset_index(name="Count")

# Create the figure
plt.figure(figsize=(10,6))

# Setting colors, creating the bar plot
p = sns.color_palette("rocket", n_colors=8)
sns.barplot(data=songs_service_count, x="pv_services", y="Count", hue="Count", pale

plt.title("Number of songs per PV service")
plt.xlabel("PV Services")
plt.ylabel("Number of songs per service")

plt.show()
```



YouTube kind of wins by a landslide in terms of number of songs... since that's a LOT of songs. I can't imagine listening to that many songs in a year I don't think, though maybe my

Spotify Wrapped can confirm that for me.

It has even more songs than NND and that was the main hub for VOCALOID content in the early days.

Given that the average ratings for Vimeo, Piapro, and SoundCloud were all very high in the previous chart, we can assume that those platforms are frequently/mainly used to redistribute popular songs to multiple websites.

You might notice that even though YouTube has way more songs added to their website, that it has a noticeably lower average rating compared to NND that has less videos. This could be due to a couple of things:

1. The most popular fan favorite songs are older songs originally from NND
2. Fans have a heavy bias towards the Japanese language + nostalgia
3. Having more and being popular isn't always a good thing apparently
    - YouTube may have more videos, but a lot of those videos could have lower ratings that ultimately drag down YouTube's average song rating

This does establish that if you want to search for the most popular songs and know where to go, YouTube will be your best bet. And even though NND is entirely in Japanese, given the amount of songs and how high their average rating is, it's still worth considering clicking around!

A Different Way to Look at Popularity (Rating vs. Frequency)

I've tried a lot of different ways to plot out the data, but one of the most interesting is plotting out all of the songs and comparing between song ratings and setlist frequency.

This is related to the heat map we created earlier near the beginning of this section. The heat map showed us something fundamentally important about the data and it's that there is a real correlation between the rating of a song and how often it shows up in concert setlists. Rather than sticking to using just the heat map to prove this, why don't we visualize it in other ways and see specific song titles as well?

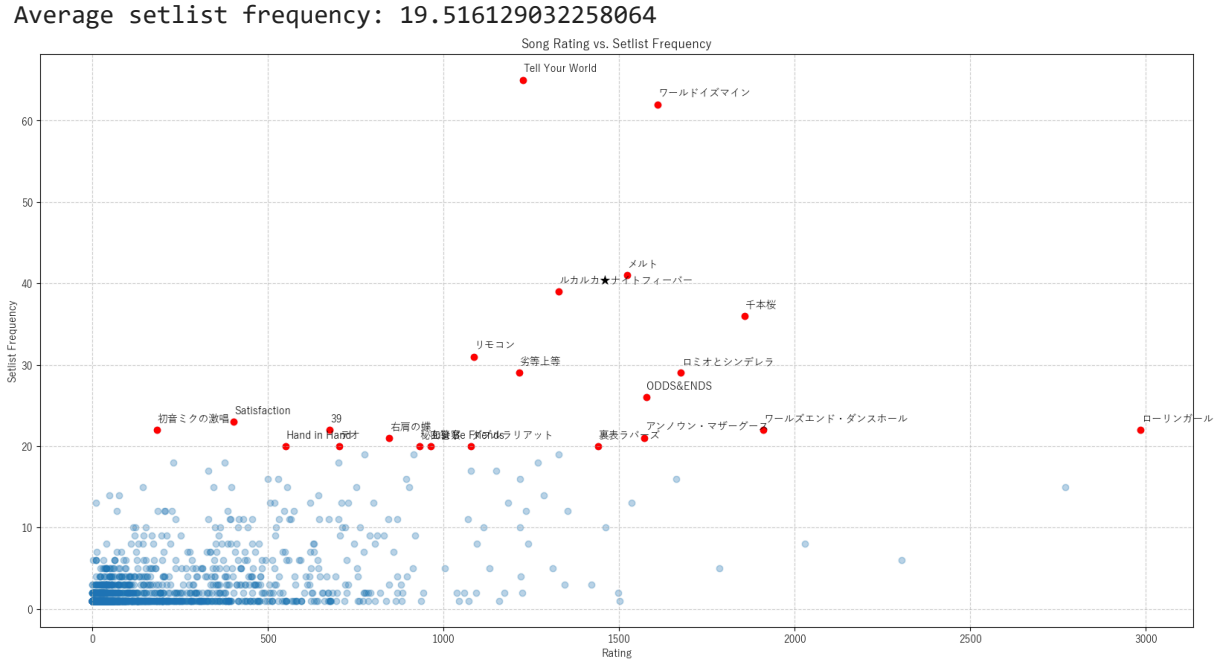
By plotting out all of our song data using a scatter plot, we can visualize all of the different songs much easier and see how dispersed the songs are. My implementation for this scatterplot is a little complicated, so I'll leave lots of comments to explain every step.

I also had to do something incredibly funny and weird to get the Japanese titles to display. Given that not everyone uses Windows I had to make sure it would load properly regardless of what OS you're running. If the fonts still don't render correctly for whatever reason please consult the PDF for the output!

In [43]:
```
# Dictionary for specific fonts based on operating system
os_specific_font = {"Windows": "Yu Gothic", "Darwin": "Hiragino Sans", "Linux": "No
```

```
# Apply the specific font based on viewer's OS so they can view the Japanese-only t
import platform
os = platform.system()
plt.rcParams["font.family"] = os_specific_font[os]

# Removing all duplicates in the setlist so we can determine the average setlist fr
drop_setlist_dupes = all_clean_songs_df.drop_duplicates(subset=["setlist_frequency"
avg_setlist_freq = drop_setlist_dupes["setlist_frequency"].mean()
print(f"Average setlist frequency: {avg_setlist_freq}")

# Create the figure
plt.figure(figsize=(20,10))

# Use a scatter plot, comparing rating with setlist_frequency
plot = sns.scatterplot()
plot.scatter(all_clean_songs_df["rating"], all_clean_songs_df["setlist_frequency"],

# Create an extra collection of plots colored differently to represent the songs th
highlight = all_clean_songs_df[all_clean_songs_df["setlist_frequency"] > avg_setlis
plot.scatter(highlight["rating"], highlight["setlist_frequency"], color="red")

# For labeling songs thay play more than the average frequency
for _,row in highlight.iterrows():
    ### Uncomment this for English titles
    #title = row["english_title"]
    #if(row["english_title"] == "None"):
    #    title = row["title"]
    plot.text(row["rating"] + 1, row["setlist_frequency"] + 1, row["title"])

plt.title("Song Rating vs. Setlist Frequency")
plt.xlabel("Rating")
plt.ylabel("Setlist Frequency")
plt.grid(True, linestyle="--", alpha=0.6)

plt.show()
```

```
Average setlist frequency: 19.516129032258064
```



Most of the songs are gathered around the bottom left corner, having both low ratings and low setlist frequency.

But as we look more towards either direction (up or right), the higher the rating/setlist frequency is the more dispersed the data points get. There are a lot less songs as we start looking further out and the data is a lot less concentrated.

What's interesting is that the songs that play more than the average frequency vary greatly in terms of rating. Most are within the range that we'd expect (~1000 and above) but there are some entries that are unexpected:

- Some songs have extremely low ratings despite being played above average amount of times ("初音ミクの激唱" and "Satisfaction")
- "ローリンガール" has the highest rating of all songs but is not played as much as "Tell Your World" even though "Tell Your World" has only half the amount of ratings
  - This one is more expected given the context that wowaka (the producer) passed away rather tragically in 2019, and usually if any of his songs are played they are a lot more emotionally impactful for fans who've been familiar with his work for as long time. This may also explain the incredibly high rating as well for this song
- "Tell Your World" is played more often than "ワールドイズマイン", which is surprising given that it's an incredibly iconic song for Hatsune Miku and the fandom considers it the #1 song everyone should know

Even if a song having a higher rating corresponds with having a higher setlist frequency, correlation does not mean causation. They're two different things, and we can see here that even if there is a pattern, the pattern doesn't necessarily guarantee that a song being popular will play more and vice versa.

Below is the list of all songs that played more often than the average setlist frequency. I'm used to reading the Japanese names, but the English names are also available below!

In [44]:
```
top_place_ratings = all_clean_songs_df[all_clean_songs_df["setlist_frequency"] > av

top_place_ratings.sort_values(by="rating", ascending=False)
```

Out[44]:

	title	english_title	artist	length_seconds	setlist_frequen
123	ローリンガール	Rollin' Girl	wowaka feat. 初音ミク	196	
74	ワールズエンド・ダンスホール	World's End Dancehall	wowaka feat. 初音ミク, 巡音ルカ	218	
64	千本桜	A Thousand Cherry Blossoms	黒うさP feat. 初音ミク	246	
80	ロミオとシンデレラ	Romeo and Cinderella	doriko feat. 初音ミク	280	
152	ワールドイズマイン	World is Mine	ryo, supercell feat. 初音ミク	255	
368	ODDS&ENDS	None	ryo feat. 初音ミク Append (Unknown)	354	
159	アンノウン・マザーグース	Unknown Mother-Goose	wowaka, ヒトリエ feat. 初音ミク V4X (Dark)	285	
83	メルト	Melt	ryo feat. 初音ミク	256	
77	裏表ラバーズ	Two-Sided Lovers	wowaka feat. 初音ミク	189	
73	ルカルカ★ナイトフィーバー	Luka Luka★Night Fever	samfree feat. 巡音ルカ	239	
268	Tell Your World	None	kz feat. 初音ミク	257	
189	劣等上等	BRING IT ON	Giga feat. 鏡音リン, 鏡音レン	235	
149	リモコン	Remote Control	じーざすP, WONDERFUL★OPPORTUNITY! feat. 鏡音リン, 鏡音レン	312	
412	ダブルラリアット	Double Lariat	アゴアニキ feat. 巡音ルカ	207	
656	Just Be Friends	None	Dixie Flatline feat. 巡音ルカ	304	

	title	english_title	artist	length_seconds	setlist_frequer
641	秘密警察	Secret Police	ぶりるP feat. 初音ミク	211	
600	右肩の蝶	Butterfly on Your Right Shoulder	のりぴー feat. 鏡音レン	265	
59	テオ	Teo	Kimura, Sakurai, Omoi feat. 初音ミク	209	
286	39	None	sasakure.UK, DECO*27 feat. 初音ミク	220	
84	Hand in Hand	None	kz feat. 初音ミク	320	
120	Satisfaction	None	kz feat. 初音ミク	201	
562	初音ミクの激唱	The Intense Voice of Hatsune Miku	cosMo@暴走P, Storyteller feat. 初音ミク	166	

Thank you!